

Camera功耗分析指导手册

WWW.UNISOC.COM

修改历史

版本号	日期	注释
V1.3	2025.06.20	新增加功耗优化案例
V1.2	2023/9/27	● 增加Android 14相关内容。 ● 增加设置功耗等级相关说明。
V1.1	2023/6/8	调整文档结构，优化文档内容。
V1.0	2022/10/9	第一次正式发布。

关键字

关键字：Camera、功耗

目录



01 测试方法

02 分析思路

03 优化措施

04 优化案例

05 参考文档

06 附录

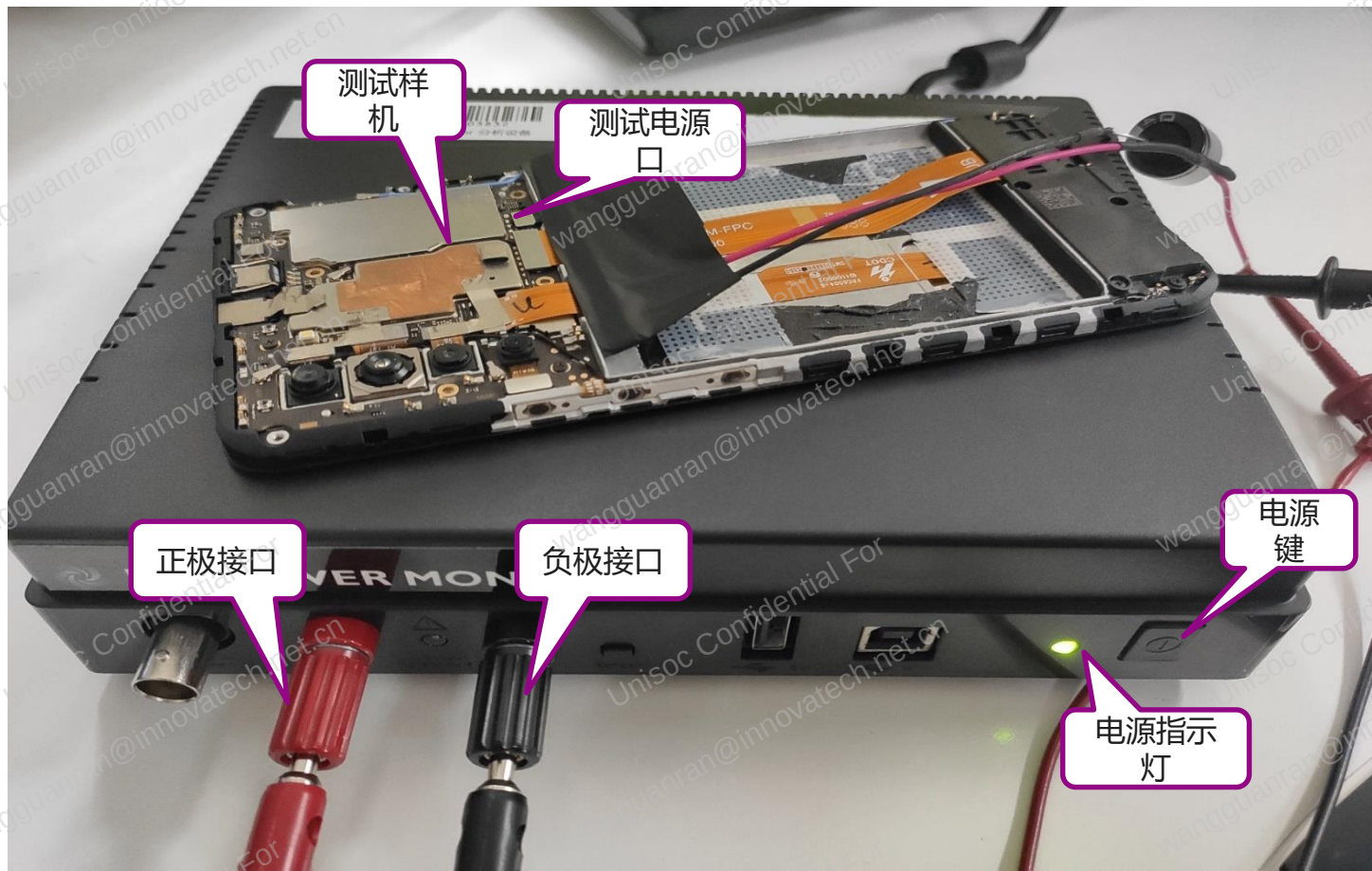
01

测试方法

测试设备

硬件设备：POWER MONITOR

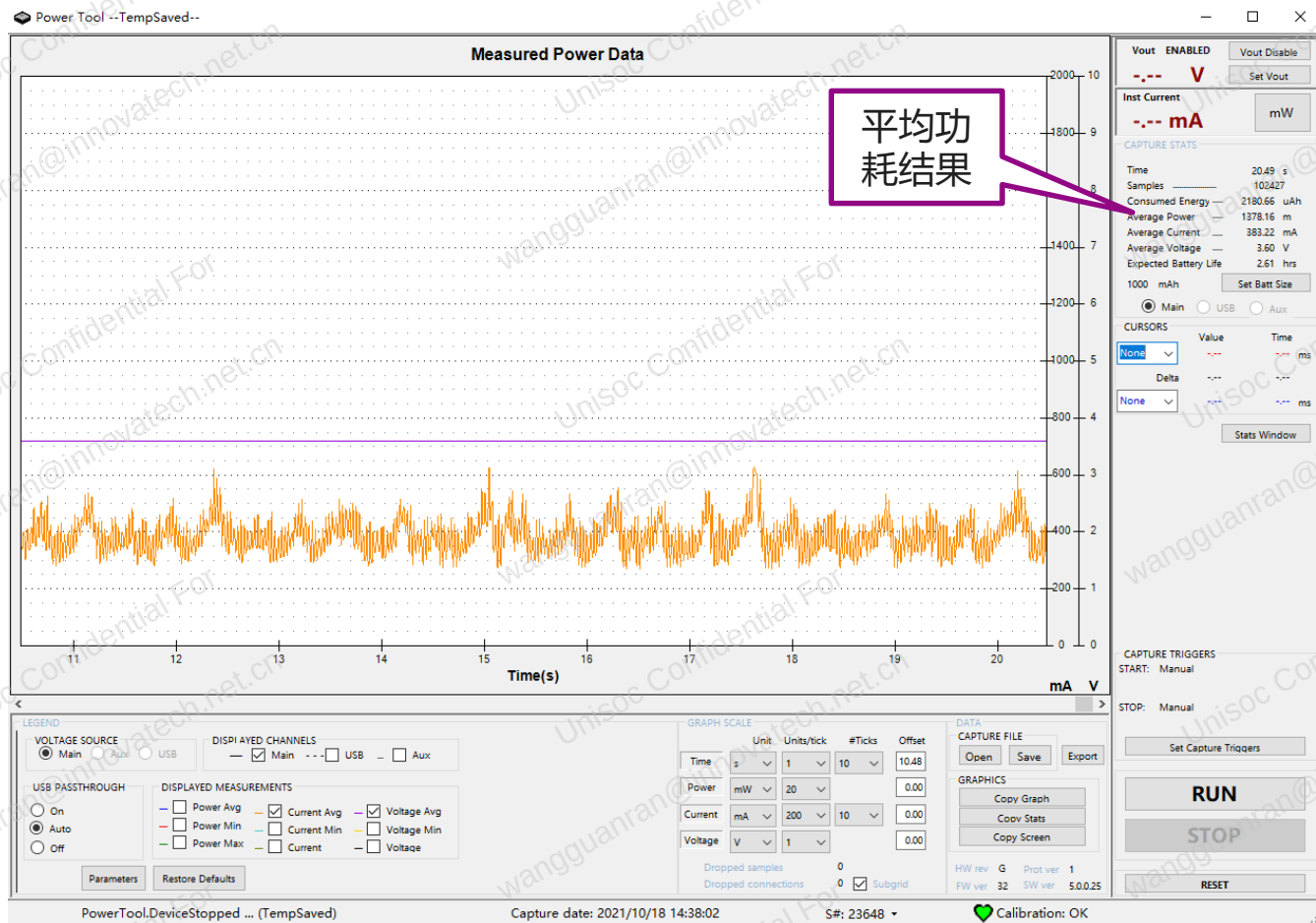
POWER MONITOR通过正负极接线连接测试样机，给测试样机供电。



测试软件

测试软件：Power Tool

控制POWER MONITOR的输出电压，同时记录电流值。



前置条件

- 设备版本

推荐使用user版本测试功耗数据（注意事项请参考[附录](#)）。如果本地调试需要push库，可临时使用debug版本，但要关闭YLog，最终测试结果以user版本为准。

注意：Android 13~16上为了提高debug版本Camera的性能，会将Camera场景的CPU频点提到最高。若在Android 13~16上使用debug版本测试Camera的功耗，需先调整CPU到正常的低功耗模式，命令如下：

```
adb shell setprop persist.vendor.cam.ppm.lowp 1
```

- 设备各功能设置

功能	飞行模式	Wi-Fi	蓝牙	数据连接	自动旋转屏幕	屏幕亮度	屏幕亮度自动调节	GPS	背光省电
设置	开启	关闭	关闭	关闭	关闭	最低	关闭	关闭	关闭

- Camera模式设置

输入*##83781#*##进入工程模式，选择HARDWARE，关闭Camera Idle Sleep，开启Camera Power Test Mode。

- 自动灭屏时间

自动灭屏时间设置为30分钟。

测试方法

以自然光下，测试后置拍照模式预览的功耗为例进行说明，具体的测试方法如下：

1. 连接设备与POWER MONITOR，使用Power Tool软件给设备上电，开机。
2. 按照测试前置条件设置设备（参考[前置条件](#)），清空后台，完成后静置5分钟。
3. 进入Camera，**固定帧率为30fps**。取景画面对准静止物体，如绿植，固定设备保持场景不变。
4. 进入Camera设置，**关闭闪光灯、HDR、美颜、自动夜景增强等**所有非基础功能。
5. 进入Camera预览画面，**静置30s**后，点击Power Tool软件记录60s Camera预览的电流，重复三次，取平均值。
该平均值为最终整机自然光下后置拍照模式预览的功耗数据（整机功耗数据）。
6. 退出Camera，清空后台，界面停留在主界面上。点击Power Tool软件记录60s的电流，重复三次，取平均值。
该值为屏幕的功耗数据（LCDON功耗数据）。

Camera功耗数据=整机功耗数据-LCDON功耗数据

02

分析思路

功耗分析

功耗分析基本流程：



确认目标

1. 确认优化目标

- 如有多个优化目标，确认各目标的优先级，例如**对比机数据指标**和**客观指标的优先级**。
- 确认优化的目标对象，是整机功耗还是Camera模块功耗。

2. 确认优化目标的合理性

- 评估**客观指标**是否合理，**对比机**功耗表现**是否满足**客观指标。
- 评估测试机和对比机的硬件能力是否相当，例如CPU处理能力、sensor硬件等。

对齐测试项和测试方法

1. 确认测试项

- 确认当前测试的背景，遮黑还是自然光。
- 确认当前测试的模式，单摄、多摄，预览、录像等。
- 确认当前模式下需要开启或关闭的功能算法。

2. 固定帧率

由于本地环境亮度不稳定，在测试时需要固定帧率。自然光下固定为30fps，遮光时测试机与对比机帧率保持一致。

3. 测试方法

- 参照[前置条件](#)和测试项进行测试，例如设备与测试物体的距离需保持固定，否则对焦会使功耗发生变化。
- 测试机和对比机测试方法保持一致。

4. 测试数据

测试前将设备静置一段时间，等待设备稳定后测试。测试时多测几组数据，剔除异常数据，其余数据取平均值，以减小测试误差。

分析影响因素1/2

- 帧率

检查测试场景下的帧率，测试时将测试机和对比机客观指标帧率固定一致。

- 尺寸

检查DCAM/ISP/DPU等模块的输入输出尺寸、sensor输出尺寸、配流的尺寸等。

- 算法功能

- 检查测试机与对比机使用的各个算法版本是否一致。若不一致，在测试机上逐个关闭算法，分解算法的功耗。
- 检查测试机相比对比机是否存在额外的功能或算法。若存在，在测试机上将其关闭，并从log中确认关闭状态，再测试功耗。

- LCDON

检查测试目标是否包含亮屏时的基础耗电。

- CPU工作频率

检查cpu-loading，确认CPU每个核的工作频率。

分析影响因素2/2

- CPU负载

通过TOP命令，检查各个线程在CPU上的负载情况。

- 图层

- 建议使用DPU/GSP实现图层合成，若使用GPU实现会增加功耗。
- 每一帧图像由多层图层合成显示。图层数量增加，每一帧需要处理的数据量也会增加，导致功耗增加。

- GPU/DDR工作频率

检查GPU/DDR在各个频点的使用情况。

- 帧数据格式

检查对比机与测试机数据格式是否一致（RAW10或是RAW14）。

- 硬件

对于不同的sensor，无法保证sensor功耗的一致性，建议使用相同的sensor进行测试。

说明：CPU、GPU、DDR等常用功耗分析工具的功能及使用方法请参考《功耗分析工具介绍》。

03

优化措施

- 帧率确认，可通过log检查以下两处帧率

- 查看sensor打印帧率，例如：

12-16 23:14:25.279 495 26999 I ov32a1q : 172, ov32a1q_drv_calc_exposure: fps = 30.000862

- 计算POP帧率，通过index和时间计算POP帧率，例如：

12-16 23:14:25.573 495 26977 D cmr_prev: 14554, prev_pop_preview_buffer: cam_id=0,
fd=0x33, chn_id=0x1, valid_num=3, index=10, timestamp=0xf2f8bd7223b0

12-16 23:14:28.906 495 26977 D cmr_prev: 14554, prev_pop_preview_buffer: cam_id=0,
fd=0x27, chn_id=0x1, valid_num=3, index=110, timestamp=0xf2f9840b5a30 //计算出POP帧率为30fps

- 帧率固定方法

- 输入*##83781#* 进入工程模式，选择HARDWARE>Camera FPS，进入preview、recording设置对应的max、min帧率。

max、min值设置为一致则表示固定帧率。

- ADB命令设置

adb shell setprop persist.vendor.cam.preview.fps 3030

adb shell setprop persist.vendor.cam.record.fps 3030

- 帧率修改方法

若该场景设置的帧率范围没有与对比机对齐，建议先检查相机应用帧率值的下发范围是否对齐

(CONTROL_AE_TARGET_FPS_RANGE下发值new Range(min,max))，如应用下发已经对齐，则需调试效果参数AE模块对齐帧率。

测试时，确认屏幕亮度百分比、关闭屏幕亮度自动调节功能。测试主界面的功耗数据，以便后续测试Camera功耗时排除LCDON的功耗，只计算Camera功耗。

LCDON测试方法：

1. 正确连接设备与POWER MONITOR，给设备上电，静置5分钟。
2. 按照[前置条件](#)，对设备进行相应设置。
3. 清空后台，保持界面停留在主界面，确保当前屏幕亮度为最低。静置5分钟后，使用Power Tool软件记录60s的电流，重复三次，取平均值。

若LCDON功耗数据有差异，建议检查确认屏幕的功耗差异点，通常联系屏幕IC厂商优化。

检查尺寸相关信息，若测试机和对比机在尺寸上存在差异，需进行对应修改。

- 检查配流尺寸。以surfaceTexture为例，预览配置为1440×1080，在Camera应用修改配流时下发的预览尺寸：

```
surfaceTexture.setDefaultBufferSize(1440,1080);
```

```
mPreviewSurface = new Surface(surfaceTexture);
```

```
12-16 23:14:24.998 495 1025 I Cam3HWI : 995, configureStreams: :hal3: camId=0, prev: w=1440, h=1080, video:  
w=0, h=0, callback: w=3264, h=2448, yuv2: w=0, h=0, cap: w=3264, h=2448
```

- 检查sensor输出尺寸，如3264×2448。联系sensor原厂修改相关尺寸及设置。

```
12-16 23:14:25.042 495 27032 D cmr_mem : 379, camera_arrange_capture_buf: sn_size 3264 2448, sn_trim 0 0  
3264 2448, image_size 3264 2448, cap_size 3264 2448
```

- 检查DCAM输入输出尺寸，如输入3264×2448，输出1632×1224。联系FAE确认及修改。

```
FA203 <6> [267149.720136][12-16 23:14:24.720] DCAM_PATH: 26978 313 dcam_path_size_cfg : cfg bin path  
done. size 3264 2448 dst 1632 1224
```


算法和功能

- Camera设置界面，将测试机和对比机的功能开关保持一致。
- 测试项相应条件下，确认使用的算法版本，常用关键字“CAM_ALGO”，例如：

PDAF版本信息：PDAF_Algo_new_version: CAM_ALGO_ISP_3A_PDAF_3.0_H0_20210111_1050

如果当前算法版本差异较大，或者测试机相比对比机增加了算法，可以通过命令关闭算法。测试关闭算法前后的功耗数据，两个数据的差为该算法的功耗数据。算法关闭示例：

算法	关闭命令
PDAF	adb shell setprop persist.vendor.camera.bypass.pdaf 1 adb shell setprop persist.vendor.cam.pd.enable 0
FDV3	adb shell setprop persist.vendor.fdtop.fdv3 0
AE3.2	adb shell setprop persist.vendor.isp.ae.power_lib 1 adb shell setprop persist.vendor.cam.isp.ae.power_ctrl 1
AF	adb shell setprop persist.vendor.cam.isp.af.bypass 1
AWB	adb shell setprop debug.isp.awb.bypass 1

- 若为算法版本不同带来的功耗差异，请联系FAE确认算法功耗与优化方案。
- 若为新增算法/功能带来的功耗差异，除了联系FAE确认外，还需在优化目标中注明新增。

说明：Android 13~16上算法以Node形式集成到Pipeline上，可以通过Bypass Node来关闭算法，可参考文档《Camera ADB命令使用指导手册》设置Bypass单个Node。

CPU工作频率1/4

使用如下命令查看一段时间内的CPU loading情况：

adb root

adb shell

./vendor/bin/power/cpu/loading/cpu_loading -t 3

```
cpuN      614400  768000  962000  1144000  1228800  1482000  1536000  1716000  1820000  2002000  online  idle
cpu0      10.14%  4.11%   4.35%   7.00%   5.07%   7.73%   2.17%   4.59%   1.69%   25.36%  72.22%  27.78%
cpu1      9.43%   3.82%   4.04%   6.51%   4.71%   7.18%   2.02%   4.27%   1.57%   23.57%  67.13%  32.87%
cpu2      8.53%   3.45%   3.66%   5.89%   4.27%   6.50%   1.83%   3.86%   1.42%   21.33%  60.75%  39.25%
cpu3      7.60%   3.07%   3.26%   5.25%   3.80%   5.79%   1.63%   3.44%   1.27%   18.99%  54.08%  45.92%
cpu4      6.05%   2.45%   2.59%   4.18%   3.03%   4.61%   1.30%   2.74%   1.01%   15.14%  43.10%  56.90%
cpu5      5.52%   2.23%   2.36%   3.81%   2.76%   4.20%   1.18%   2.49%   0.92%   13.79%  39.26%  60.74%
Freq      14.05%  5.69%   6.02%   9.70%   7.02%   10.70%  3.01%   6.35%   2.34%   35.12%
sum:      4862043.77kHz
power:    0.00
cpuN      1228800  1482000  1536000  1742000  1872000  2002000  online  idle
cpu6      4.25%   2.55%   2.76%   6.16%   47.77%  0.00%   63.48%  36.52%
cpu7      4.50%   2.70%   2.93%   6.53%   50.67%  0.00%   67.34%  32.66%
Freq      6.69%   4.01%   4.35%   9.70%   75.25%  0.00%
sum:      2336608.36kHz
power:    0.00
sum_total: 7198652.13kHz
power_total: 0.00
```

从上图可知，这段时间CPU小核主要工作在2002000频点，大核主要工作在1872000频点。

整体工作在高频点，所以功耗相对会偏高。

若算法和尺寸等基本一致的情况下，测试机CPU工作频率更高，可以将测试机和对比机的CPU固定到同一频率后测试，具体方法如下：

1. 查看每个CPU核的工作频率范围

```
cat /sys/bus/cpu/devices/cpu*/cpufreq/scaling_available_frequencies
```

2. 设置cpuX的频率

```
echo userspace > /sys/bus/cpu/devices/cpuX/cpufreq/scaling_governor
```

```
echo 614400 > /sys/bus/cpu/devices/cpuX/cpufreq/scaling_setspeed //设置频率为614400
```

3. 验证，查看cpuX的当前频率

```
cat /sys/bus/cpu/devices/cpuX/cpufreq/scaling_cur_freq
```

注意：cpuX中的X代指CPU核心（如cpu6），每个核都需要单独执行命令进行相应操作。

若降低频率后功耗可以对齐，则可在代码中修改CAM_PERFORMANCE_LEVEL以降低频率，通过setCamPreformanceScene接口设置CAM_PERFORMANCE_LEVEL。检查测试机和对比机的等级，适当降低测试机的等级，可以减少功耗，但是需要考虑对性能的影响。

```
HAL_LOGD("listSize %d", listSize);
if (listSize > 0) {
    // for shot2shot
    return;
} else if (getMultiCameraMode() == MODE_BLUR ||
    getMultiCameraMode() == MODE_BOKEH ||
    getMultiCameraMode() == MODE_PORTRAIT ||
    getMultiCameraMode() == MODE_PORTRAIT_SINGLE ||
    getMultiCameraMode() == MODE_PORTRAIT_SCENE ||
    mSprdAppmodeId == CAM_SCENE_MODE_NIGHT) {
    setCamPerformanceScene(CAM_PERFORMANCE_LEVEL_4);
} else if (mRecordingMode == true) {
    setCamPerformanceScene(CAM_PERFORMANCE_LEVEL_3);
} else if (BurstCapCnt > 1) {
    setCamPerformanceScene(CAM_PERFORMANCE_LEVEL_6);
} else {
    setCamPerformanceScene(CAM_PERFORMANCE_LEVEL_1);
}
```

Android 13~14设置功耗等级可参考[附录](#)，
Android 15~16的代码路径有差异，整体可以参考Android 14的。

通过 “powerhal” 关键字确认当前场景进入的powerhint场景，修改该场景下的配置，限制CPU的频率，但需要注意此修改会影响性能，需要依据实际情况修改。例如修改camera_lowpower场景：

```
M3613B2 12-16 23:14:26.477 516 516 D PowerHAL: ###Enter camera_lowpower scene bgn###
```

文件：power_scene_config.xml

根据scene name修改相应场景配置，限制CPU的工作频率范围：

```
<scene name="camera_lowpower" >  
<set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_min_freq" value="768000" />  
<set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_max_freq" value="1100000" />  
<set path="/sys/devices/system/cpu/cpufreq/policy4" file="scaling_min_freq" value="768000" />  
<set path="/sys/devices/system/cpu/cpufreq/policy4" file="scaling_max_freq" value="1400000" />  
</scene>
```

- policy：对应大小核。

示例为4+4的配置，大核从4开始，policy0表示小核，policy4表示大核。

若为6+2配置，大核从6开始，则policy6配置大核。

- scaling_min_freq：最小频率。
- scaling_max_freq：最大频率。

CPU负载1/2

1. 查看各个进程CPU的负载情况，确认是否存在CPU负载很高的异常进程。例如：

adb root

adb shell

ums512_1h10:/ # top

Tasks: 549 total, 3 running, 546 sleeping, 0 stopped, 0 zombie											
Mem: 3610988K total, 3553216K used, 57772K free, 2240K buffers											
Swap: 2238808K total, 629992K used, 1608816K free, 1040784K cached											
300%cpu 108%user 9%nice 201%sys 463%idle 0%iow 12%irq 6%siirq 0%host											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	ARGS
678	cameraserver	20	0	1.0G	266M	36M	S	155	7.5	3:39.09	android.hardware.camera.provider@2.4-service
301	logd	30	10	10G	7.1M	3.5M	S	60.6	0.2	17:14.76	logd
835	root	20	0	11G	4.3M	4.1M	S	19.0	0.1	2:51.62	collect_apr
1170	system	18	-2	19G	293M	276M	S	16.3	8.3	14:58.66	system_server
581	system	20	0	10G	2.4M	2.2M	S	13.3	0.0	2:01.90	ylogkat
764	system	-3	-8	11G	38M	30M	R	10.3	1.0	1:50.95	surfaceflinger
9867	u0_a147	10	10	16G	181M	120M	R	10.0	5.1	0:10.90	com.android.camera2
848	cameraserver	20	0	11G	14M	11M	S	7.3	0.4	0:16.90	cameraserver
687	system	-3	0	11G	14M	14M	S	6.0	0.4	0:50.15	android.hardware.graphics.composer@2.1-service
561	system	20	0	11G	2.5M	2.5M	S	5.3	0.0	26:23.05	ylog
175	root	RT	0	0	0	0	S	4.3	0.0	11:41.86	[sugov:0]
906	system	20	0	11G	3.8M	2.8M	S	4.0	0.1	76:21.51	slogmodem
10224	root	20	0	0	0	0	S	3.0	0.0	0:02.36	[dcam0_interrupt]
703	system	20	0	10G	3.4M	3.2M	S	2.6	0.0	0:06.87	android.hardware.sensors-service.multihal

其中，CPU负载最高的进程是cameraserver。

2. 查看camera provider进程下各线程的负载情况，检查是否存在负载较高的异常线程，进行优化。

ums512_1h10:/ # **top -H -p 678** //678是camera provider的进程号

Threads: 266 total, 4 running, 262 sleeping, 0 stopped, 0 zombie
Mem: 3610988K total, 3561860K used, 49128K free, 2272K buffers
Swap: 2238808K total, 655848K used, 1582960K free, 1070084K cached
800%cpu 110%user 12%nice 202%sys 458%idle 0%iow 12%irq 7%irq 0%host

TID	USER	PR	NI	VIRT	RES	SHR	S	[%CPU]	%MEM	TIME+	THREAD	PROCESS
10963	cameraserver	10	-10	1.0G	257M	36M	S	28.0	7.2	0:05.60	async_FaceDetec	android.hardware.camera.provider@2.4-service
10706	cameraserver	10	-10	1.0G	257M	36M	R	20.3	7.2	0:03.71	ispmw2	android.hardware.camera.provider@2.4-service
10731	cameraserver	20	0	1.0G	257M	36M	S	12.6	7.2	0:02.23	lscctrl	android.hardware.camera.provider@2.4-service
10726	cameraserver	10	-10	1.0G	257M	36M	S	10.0	7.2	0:01.87	aectrl	android.hardware.camera.provider@2.4-service
10902	cameraserver	0	-20	1.0G	257M	36M	S	7.3	7.2	0:01.34	SinkNode	android.hardware.camera.provider@2.4-service
10741	cameraserver	20	0	1.0G	257M	36M	S	7.3	7.2	0:01.39	awbctrl	android.hardware.camera.provider@2.4-service
10862	cameraserver	20	0	1.0G	257M	36M	R	7.0	7.2	0:01.31	TW_Preview	android.hardware.camera.provider@2.4-service
10729	cameraserver	20	0	1.0G	257M	36M	S	5.0	7.2	0:00.90	afctrl	android.hardware.camera.provider@2.4-service
10668	cameraserver	0	-20	1.0G	257M	36M	S	5.0	7.2	0:00.88	pmg_thread	android.hardware.camera.provider@2.4-service
7316	cameraserver	20	0	1.0G	257M	36M	S	3.6	7.2	0:05.10	CapRequest_0	android.hardware.camera.provider@2.4-service
10801	cameraserver	20	0	1.0G	257M	36M	S	3.3	7.2	0:00.56	gyro	android.hardware.camera.provider@2.4-service

图层

检查测试场景下的图层信息：

adb shell dumpsys SurfaceFlinger

例如，该场景下有三层图层，都是由DPU合成。

```
Display 0 HWC layers:
```

Layer name	Z	Window Type	Comp Type	Transform	Disp Frame (LTRB)	Source Crop (LTRB)	Frame Rate (Explicit) [Focused]
SurfaceView - com.android.camera2/com.android.camera.CameraLauncher#1	rel -2	0	DEVICE	0x5	0 96 1080 1536	0.0 0.0 1440.0 1080.0	[*]
com.android.camera2/com.android.camera.CameraLauncher#0	rel 0	1	DEVICE	0	0 0 1080 2160	0.0 0.0 1080.0 2160.0	[*]
NavigationBar0#0	rel 0	2019	DEVICE	0	0 2016 1080 2160	0.0 0.0 1080.0 144.0	[]


```
h/w composer state:
h/w composer enabled
```

comp type	format	fbc	pitch	height	transform	blend	alpha	zorder	dl	dt	db
DPU	OTHER	N	1440	1080	90&FLIP_H	NONE	255	0			
DPU	RGBA8888	Y	1088	2160	0	PREMULT	255	1	0	0	0
DPU	RGBA8888	Y	1088	144	0	PREMULT	255	2	0	0	0

图层名称

图层合成方式

说明：各个平台修改图层方案并不通用，若为图层引起的功耗问题，请联系FAE支持。

GPU工作频率

检查和固定GPU的工作频率：

1. 查看GPU的工作频率范围

```
cat /sys/class/devfreq/600000000.gpu/available_frequencies
```

2. 修改GPU的工作频率

```
echo 550000000 > /sys/class/devfreq/600000000.gpu/max_freq
```

```
echo 550000000 > /sys/class/devfreq/600000000.gpu/min_freq //将GPU频率固定在550MHz
```

3. 验证修改是否成功

```
cat /sys/class/devfreq/600000000.gpu/max_freq
```

```
cat /sys/class/devfreq/600000000.gpu/min_freq
```

4. 查看当前GPU工作频率是否固定为设置的值

```
cat /sys/class/devfreq/600000000.gpu/cur_freq
```

注意：项目不同，“.gpu”文件前面的数字也不同（如600000000.gpu），需要进入/sys/class/devfreq/目录确认。

若GPU工作频率异常，请联系FAE确认及修改。

DDR工作频率

检查和固定DDR的工作频率(建议先检查频率范围后再修改工作频率):

1. 查看DDR的工作频率范围

```
cat /sys/class/devfreq/scene-frequency/sprd-governor/DDRinfo_freq_table
```

2. 修改DDR的工作频率

```
echo 933 > /sys/class/devfreq/scene-frequency/sprd_governor/scaling_force_ddr_freq //强制将DDR频率固定在933MHz
```

3. 查看当前DDR的工作频率，验证修改是否成功

```
cat /sys/class/devfreq/scene-frequency/sprd-governor/DDRinfo_cur_freq
```

若DDR工作频率异常，请联系FAE确认及修改。

04

优化案例

现象描述

720p录像模式，预览和录像功耗对比机高。

	桌面 (mA)	720P预览 (mA)	720P录像 (mA)
测试机，飞行模式，user版本	128	418	477
对比机，飞行模式，user版本	112	282	313
去除桌面电流的功耗GAP		120	148

从上表来看，去除桌面电流后，720P预览高了120mA，720P录像高了148mA。这个功耗明显偏高，首先要确认CPU的工作状态，查看CPU是不是跑到最高频点，接着查跑了哪些算法，这些算法是否可以关闭等。

排查流程

1. **基本信息确认**：确认软硬件的差异、基础信息对齐
2. **CPU**：CPU Loading
3. **Sensor**：寄存器配置、三路供电、FPS限制
4. **DDR**：DDR Loading
5. **GPU/DPU**：图层合成方式
6. **Camera HAL**：手势识别、人脸检测、美颜、EIS、AI场景、场景识别等
7. **Camera效果**：PDAF、LSC、AWB
8. **音视频编码**
9. **硬件功耗拆解**
10. **功耗收益**

基本信息确认

结合对比机的平台信息对比测试，尽量在有同等或类似的硬件能力平台上进行比较。安卓版本、DDR等硬件的不同会导致一定的功耗差异，在设定功耗目标值的时候需要考虑到这些因素。

测试机对比机都使用USER+ROOT版本测试，并将ro.debuggable设为0（当此属性为1时，测试功耗需要设置“adb shell setprop persist.vendor.cam.ppm.lowp 1”）。

	参数	测试机	对比机
软件平台	OS	Android 13	Android 13
	Kernel	kernel5.4	kernel5.4
	测试版本	USER+ROOT	USER+ROOT
网络模式	通信制式	飞行模式	飞行模式
主芯片	工艺制程	6nm	6nm
	CPU	2.7Ghz A76*1+ 2.3GHz A76*3+ 2.1GHz A55*4	2.7Ghz Kryo670*1+ 2.4GHz Kryo670*3+ 1.9GHz Kryo670*4
	GPU	Arm® Mali™-G57 4cores, up to 850MHz	Adreno 642L 812 MHz
	DDR	LPDDR4X	LPDDR4X
Memory	RAM	4G	4G
	ROM	64G	64G
	Flash	UFS3.1	UFS2.2
	文件系统	f2fs	f2fs
Sensor	SIZE	2688×1512	2688×1512
	AF	定焦模组	定焦模组

1. 抓取10s预览时CPU loading信息。

```
adb shell /vendor/bin/power/cpu/loading/cpu_loading -t 10
```

2. 打印信息显示全核都是以最高频率在运行，另检查ylog打印信息，在打开camera后没有载入camera_lowpower策略。而正常情况下，在打开相机时因对性能要求高，camera会开启高频策略（如camera_high_perf），并在sensor出35帧（由CAM_POWERHINT_WAIT_COUNT定义）后载入低频策略（如camera_lowpower）降低功耗。（各电源策略可在对应平台的power_scene_config.xml文件中配置）
3. 这个问题使用的是三方App，当前基线会将不在topAppList的三方App都进行提频处理。故需要把这个三方App添加到topAppList中，并在设置预览功耗策略的时候针对这个三方App进行配置。这样做的益处是不会影响其他的场景，只针对这个App进行设置。

例如在adjustPreviewPerformance函数return前判断top_app_id，当是这个三方App时设置CAM_PERFORMANCE_LEVEL_1，这样修改后，这个三方App在进入预览后就会载入camera_lowpower策略。将特定应用添加topAppList可参考附录[添加TOPAPP](#)，正常流程打印如下：

```
//开相机
```

```
M2DC3CA 04-12 12:19:29.545 765 16194 I Camera2ClientBase: Camera 0: Opened. Client: com.test.camera (PID 12354, UID 10176)
```

```
//进入开相机camera_high_perf策略
```

```
M2DC527 04-12 12:19:29.554 626 626 D PowerHAL: ###Enter camera_high_perf scene bgn###
```

```
//预览出35帧后退出camera_high_perf，进入览camera_lowpower
```

```
M2E0536 04-12 12:19:31.202 626 626 D PowerHAL: ###Exit camera_high_perf scene bgn###
```

```
M2E0552 04-12 12:19:31.203 626 626 D PowerHAL: ###Enter camera_lowpower scene bgn###
```


- | 去除桌面电流 | 720P预览 (mA) | 720P录像 (mA) |
|-----------------------|-------------|-------------|
| 测试机-原始状态 | 290 | 349 |
| 测试机-载入camera_lowpower | 212 | 232 |
| camera_lowpower带来的优化 | 88 | 117 |

[illegible]

- ```
<scene name="camera_lowpower" >
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_min_freq" value="614400" />
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_max_freq" value="768000" />
 <set path="/sys/devices/system/cpu/cpu4" file="online" value="0" />
 <set path="/sys/devices/system/cpu/cpu5" file="online" value="0" />
 <set path="/sys/devices/system/cpu/cpu6" file="online" value="0" />
 <set path="/sys/devices/system/cpu/cpu7" file="online" value="0" />
</scene>
```

cpuN	614400	768000	936000	1105000	1228800	1404000	1560000	1703000	1846000	2002000	2184000	online	idle
cpu0	22.66%	36.05%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	58.71%	41.29%
cpu1	22.87%	36.38%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	59.26%	40.74%
cpu2	23.42%	37.26%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	60.68%	39.32%
cpu3	22.59%	35.93%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	58.51%	41.49%
Freq	38.60%	61.40%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%		
sum:	1680749.20kHz												
power:	0.00												

cpuN	614400	768000	936000	1105000	1228800	1378000	1536000	1703000	1872000	2041000	2210000	2301000	online	idle
cpu4	unplugged													
cpu5	unplugged													
cpu6	unplugged													
Freq	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%		
sum:	0.00kHz													
power:	0.00													

cpuN	614400	768000	1228800	1536000	1820000	2002000	2132000	2262000	2366000	2457000	2548000	2704000	online	idle
cpu7	unplugged													
Freq	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%		
sum:	0.00kHz													
power:	0.00													

sum_total:	1680749.20kHz												
power_total:	0.00												

## 提频积极性

调频时，计算目标频率的方式是 $\text{max\_freq} \times \text{cpu使用率}$ 。引入 $\text{freq\_margin}$ 调频参数后，假设 $\text{freq\_margin}$ 是30，计算方式就变成了 $\text{max\_freq} \times (1 + 30\%) \times \text{cpu使用率}$ ， $\text{freq\_margin}$ 的取值范围是 $[-100, 100]$ 。

```
<set path="/sys/devices/system/cpu/cpufreq/policy0/schedutil" file="freq_margin" value="-50" />
<set path="/sys/devices/system/cpu/cpufreq/policy4/schedutil" file="freq_margin" value="-60" />
<set path="/sys/devices/system/cpu/cpufreq/policy7/schedutil" file="freq_margin" value="-60" />
```

其中：

- policy0为小核CPU0~3
- policy4为中核CPU4~6
- policy7为大核CPU7

不同平台的节点路径不同，例如有部分平台：

- policy0为小核
- policy6为大核

具体情况可在以下目录下ls查看

```
adb root
adb remount
adb shell
cd /sys/devices/system/cpu/cpufreq
ls
```

1. 寄存器配置

从log中确认是使用的2688×1512 full size的这一组setting，且寄存器配置与对比机一致。log关键词为：

prev\_get\_sn\_capture\_mode|sensor\_set\_mode|prev\_pre\_set|prev\_get\_sn\_preview\_mode|updateSensorMode。

M009890 04-30 11:51:29.946 378 4222 D cmr\_prev: 6573, prev\_get\_sn\_preview\_mode: candidate mode=2, height = 1512, width = 2688, is\_high\_fps=0 mode\_max\_fps=0  
M0098F8 04-30 11:51:29.950 378 4201 I sns\_drv\_u: 2814, sensor\_set\_mode\_msg: mode = 2.//这里的mode指sensor驱动中配置的第2组setting(从0开始计数)

2. AVDD\DVDD\IOVDD

供电电压正常、电流与对比机较一致。

	intention Used	Voltage(V)	Current(mA)
VDDCAMA0	后主模拟电	2.82	28.4
VDDCAMD0	后主数字电	1.24	21.4
VDDCAMIO	Camera IO	1.88	0

3. FPS

测试机对比机都固定25fps测试。

➢ 展锐平台固定25fps方式：

adb shell setprop persist.vendor.cam.preview.fps 2525 //普通拍照模式预览帧率

adb shell setprop persist.vendor.cam.record.fps 2525 //录像模式帧率

➢ 帧率设置说明：如 persist.vendor.cam.preview.fps 1510 //即设置帧率范围10~15

## 1. 抓取10s预览时DDR loading信息。

```
adb shell /vendor/bin/power/ddr/loading/ddr_loading -t 10
```

➤ 默认情况下，camera场景会把DDR频点提频到1866M。

```
533: 0.00%
768: 0.00%
1066: 0.00%
1244: 0.00%
1536: 0.00%
1866: 100.00%
unit:10ms
theory_bw = 0 4264000 6144000 8528000 9952000 12288000 14928000
overflow = 1075200 1492400 2150400 2984800 3483200 4300800 5224800 4294967295
underflow = 0 1392400 2050400 2884800 3383200 4200800 5124800 5872400
```

➤ 设置CAM\_LOW的DFS后，DDR频点降至1536M。

```
DDRFreq 384Mhz 533Mhz 768Mhz 1066Mhz 1244Mhz 1536Mhz 1866Mhz 0Mhz
FreqRatio 0.00% 0.00% 4.17% 0.73% 0.17% 93.85% 1.08% 0.00%

DDRState idle_time: write_time: read_time: sref_time: light_time:
StateRatio 40.25% 25.28% 31.22% 2.30% 0.96%
```

➤ camera不设置DFS后，DDR频点主要跑在768M，720P预览功耗下降10mA。

```
DDRFreq 384Mhz 533Mhz 768Mhz 1066Mhz 1244Mhz 1536Mhz 1866Mhz 0Mhz
FreqRatio 0.00% 0.00% 86.94% 11.98% 1.08% 0.00% 0.00% 0.00%

DDRState idle_time: write_time: read_time: sref_time: light_time:
StateRatio 29.67% 28.38% 33.01% 6.09% 2.86%
```

## 2. 因当camera不设置DFS的方案能够满足当前的使用场景，故前采用此方案。以UMS9620平台为例，camera不设置DFS方案如下：

在文件device/sprd/mpool/module/camera/msoc/qogim6pro/qogim6pro.mk中，配置CONFIG\_CAMERA\_DFS\_FIXED\_MAXLEVEL := 1。



# GPU/DPU

查看[图层](#)合成方式:

dumpsys SurfaceFlinger得到图层信息，确认与对比机同步使用DPU进行合成。

# Camera HAL

HAL中集成有手势识别、人脸检测、美颜、EIS、AI场景、场景识别等功能，可以按项目需求进行裁剪。本案例不需要这些功能，所以可以直接删掉LWP中所有node后测试。也可使用如下lwpbypass命令bypass lwp系统中的所有node处理，进行快速测试，设置后重启相机即可生效。

adb shell setprop persist.vendor.cam.lwpbypass.all 1

```
Threads: 126 total, 6 running, 120 sleeping, 0 stopped, 0 zombie
Mem: 3462964K total, 3334636K used, 128328K free, 26164K buffers
Swap: 2147032K total, 14908K used, 2132124K free, 1697712K cached
800%cpu 162%user 13%nice 104%sys 512%idle 0%iow 8%irq 2%sig 0%host
```

TID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	THREAD	PROCESS
3360	cameraserver	20	0	465M	127M	40M	R	38.4	3.7	0:00.42	lscctrl	android.hardware.
3479	cameraserver	10	-10	465M	127M	40M	S	32.6	3.7	0:00.47	async_FaceDetect	android.hardware.
3388	cameraserver	20	0	465M	127M	40M	S	17.3	3.7	0:00.27	awbctrl	android.hardware.
3372	cameraserver	-5	0	465M	127M	40M	S	13.4	3.7	0:00.23	ispmw2	android.hardware.
3347	cameraserver	RT	0	465M	127M	40M	R	11.5	3.7	0:00.23	pmg_thread	android.hardware.
3469	cameraserver	20	0	465M	127M	40M	S	5.7	3.7	0:00.07	singleFrameStrea	android.hardware.
3443	cameraserver	20	0	465M	127M	40M	R	5.7	3.7	0:00.09	SinkNode	android.hardware.
3365	cameraserver	RT	0	465M	127M	40M	S	5.7	3.7	0:00.11	aectrl	android.hardware.
3459	cameraserver	20	0	465M	127M	40M	R	3.8	3.7	0:00.05	SinkNode	android.hardware.
3377	cameraserver	20	0	465M	127M	40M	S	3.8	3.7	0:00.04	aflctrl	android.hardware.
3329	cameraserver	20	0	465M	127M	40M	S	3.8	3.7	0:00.07	prev_assist	android.hardware.
3324	cameraserver	10	-10	465M	127M	40M	S	3.8	3.7	0:00.05	grab	android.hardware.
3016	cameraserver	20	0	465M	127M	40M	S	3.8	3.7	0:04.76	CapRequest_0	android.hardware.
2763	cameraserver	20	0	465M	127M	40M	S	3.8	3.7	0:05.52	CapRequest_0	android.hardware.
3441	cameraserver	20	0	465M	127M	40M	S	1.9	3.7	0:00.05	Camera-0-FrameD	android.hardware.
3445	cameraserver	20	0	465M	127M	40M	S	1.9	3.7	0:00.01	third_party_rou	android.hardware.
3367	cameraserver	20	0	465M	127M	40M	S	1.9	3.7	0:00.02	ispmw0	android.hardware.
3353	cameraserver	20	0	465M	127M	40M	R	1.9	3.7	0:00.01	gyro	android.hardware.
3375	cameraserver	20	0	465M	127M	40M	S	1.9	3.7	0:00.01	ispsof	android.hardware.
3480	cameraserver	10	-10	465M	127M	40M	S	0.0	3.7	0:00.00	cap_FaceDetect	android.hardware.
3455	cameraserver	20	0	465M	127M	40M	S	0.0	3.7	0:00.00	scale	android.hardware.
3454	cameraserver	20	0	465M	127M	40M	S	0.0	3.7	0:00.00	cap_MENR5	android.hardware.

Bypass LWP前camera各线程

```
Threads: 126 total, 5 running, 121 sleeping, 0 stopped, 0 zombie
Mem: 3462964K total, 3344872K used, 118092K free, 52904K buffers
Swap: 2147032K total, 152496K used, 1994536K free, 1711936K cached
800%cpu 127%user 19%nice 94%sys 550%idle 0%iow 8%irq 2%sig 0%host
```

TID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	THREAD	PROCESS
4620	cameraserver	20	0	462M	123M	41M	R	35.4	3.6	0:07.72	lscctrl	android.hardware.
4625	cameraserver	20	0	462M	123M	41M	S	20.8	3.6	0:04.32	awbctrl	android.hardware.
4613	cameraserver	-5	0	462M	123M	41M	S	14.5	3.6	0:02.81	ispmw2	android.hardware.
4589	cameraserver	RT	0	462M	123M	41M	S	10.4	3.6	0:02.56	pmg_thread	android.hardware.
4670	cameraserver	20	0	462M	123M	41M	S	8.3	3.6	0:01.47	SinkNode	android.hardware.
4599	cameraserver	RT	0	462M	123M	41M	S	8.3	3.6	0:01.49	aectrl	android.hardware.
4687	cameraserver	20	0	462M	123M	41M	R	4.1	3.6	0:00.68	SinkNode	android.hardware.
4668	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:00.73	Camera-0-FrameD	android.hardware.
4628	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:00.89	aflctrl	android.hardware.
4566	cameraserver	10	-10	462M	123M	41M	S	4.1	3.6	0:00.85	grab	android.hardware.
4571	cameraserver	20	0	462M	123M	41M	R	4.1	3.6	0:00.97	prev_assist	android.hardware.
3108	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:26.25	CapRequest_0	android.hardware.
4693	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.62	singleFrameStrea	android.hardware.
4672	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.23	third_party_rou	android.hardware.
4615	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.22	ispsof	android.hardware.
4608	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.37	ispmw0	android.hardware.
4609	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.01	ispmw1	android.hardware.
4584	cameraserver	10	-10	462M	123M	41M	S	2.0	3.6	0:00.06	preview_cb	android.hardware.
4554	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.13	sensor_oem	android.hardware.
2764	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:19.81	CapRequest_0	android.hardware.

Bypass LWP后camera各线程



# Camera效果

一般情况下，PDAF、LSC、AWB这三个线程占用CPU较多，可以进行隔帧处理，但是会影响图像，修改后需要效果同事测试评估。当前使用定焦模组，不支持PDAF，可对LSC、AWB进行隔帧处理。（LSC、AWB 隔帧方法见[附录](#)）

命令提示符 - adb shell

Threads: 126 total, 5 running, 121 sleeping, 0 stopped, 0 zombie  
Mem: 3462964K total, 3344872K used, 118092K free, 52904K buffers  
Swap: 2147032K total, 152496K used, 1994536K free, 1711936K cached  
800%cpu 127%user 19%nice 94%sys 550%idle 0%iow 8%irq 2%sig 0%host

TID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	THREAD	PROCESS
4620	cameraserver	20	0	462M	123M	41M	R	35.4	3.6	0:07.72	lscctrl	android.hardware.
4625	cameraserver	20	0	462M	123M	41M	S	20.8	3.6	0:04.32	awbctrl	android.hardware.
4613	cameraserver	-5	0	462M	123M	41M	S	14.5	3.6	0:02.81	ispmw0	android.hardware.
4589	cameraserver	RT	0	462M	123M	41M	S	10.4	3.6	0:02.56	pmg_thread	android.hardware.
4670	cameraserver	20	0	462M	123M	41M	S	8.3	3.6	0:01.47	SinkNode	android.hardware.
4599	cameraserver	RT	0	462M	123M	41M	S	8.3	3.6	0:01.49	aectl	android.hardware.
4687	cameraserver	20	0	462M	123M	41M	R	4.1	3.6	0:00.68	SinkNode	android.hardware.
4668	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:00.73	Camera-0-FrameD	android.hardware.
4628	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:00.89	aflctrl	android.hardware.
4566	cameraserver	20	-10	462M	123M	41M	S	4.1	3.6	0:00.85	grab	android.hardware.
4571	cameraserver	20	0	462M	123M	41M	R	4.1	3.6	0:00.97	prev_assist	android.hardware.
3108	cameraserver	20	0	462M	123M	41M	S	4.1	3.6	0:26.25	CapRequest_0	android.hardware.
4693	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.62	singleFrameStrea	android.hardware.
4672	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.23	third_party_rou	android.hardware.
4615	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.22	ispsof	android.hardware.
4608	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.37	ispmw0	android.hardware.
4609	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.01	ispmw1	android.hardware.
4584	cameraserver	10	-10	462M	123M	41M	S	2.0	3.6	0:00.06	preview_cb	android.hardware.
4554	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:00.13	sensor_oem	android.hardware.
2764	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:19.81	CapRequest_0	android.hardware.
608	cameraserver	20	0	462M	123M	41M	R	2.0	3.6	0:23.48	CapRequest_0	android.hardware.
381	cameraserver	20	0	462M	123M	41M	S	2.0	3.6	0:25.68	CapRequest_0	android.hardware.
4706	cameraserver	10	-10	462M	123M	41M	S	0.0	3.6	0:00.00	cap_FaceDetect	android.hardware.

# 音视频编码

修改media\_profiles.xml文件，测试机同步对比机的编码设置。文件路径：  
device/sprd/mpool/module/vendor/media/codec/msoc/qogirn6pro/media\_profiles.xml

## ➤ 测试机

```
<EncoderProfile quality="720p" fileFormat="mp4" duration="60">
 <Video codec="h264"
 bitRate="8000000" //比特率默认配置8M
 width="1280"
 height="720"
 frameRate="30" />
 <Audio codec="aac"
 bitRate="96000" //比特率默认配置96k
 sampleRate="44100" //采样率默认配置44.1k
 channels="1" />
</EncoderProfile>
```

## ➤ 对比机

```
<EncoderProfile quality="720p" fileFormat="mp4" duration="60">
 <Video codec="h264"
 bitRate="2000000" //比特率配置2M
 width="1280"
 height="720"
 frameRate="30" />
 <Audio codec="aac"
 bitRate="16000" //比特率配置16k
 sampleRate="8000" //采样率配置8k
 channels="1" />
</EncoderProfile>
```

# 硬件功耗分解

在上述各部分都进行优化后，如果需要详细查到每个硬件的耗电情况，则需要进行硬件功耗分解，分解出每一路电源的功耗，看其是否合理、是否有优化的可能。

			Home界面			camera录像预览720P	
	intention Used	Voltage(V)	Current(mA) @Power domain	Current(mA) @VBAT	Voltage(V)	Current(mA) @Power domain	Current(mA) @VBAT
VDDCORE(DCDCCORE)	VDDTOP(TOP, AP, Aon Sram, IPA, Analog, DDR, DDRPHY)	0.76	24.5	5.5	0.784	40	9.2
VDDGPU(DCDCGPU)	GPU Natt 4C	0	0	0	0.04	0	0
VDDSDRAM(DCDCSRAM0)	Dual Rail Sram AUDCP + ISE 的memory处于retention状态	0.76	4.8	1.1	0.788	41	9.5
DCDCCPU1	3*A76 @ 2.6G	0--0.68	3.9	0.8	0.34--0.68	37	7.4
DCDCCPU2	1*A76 @ 2.7G	0--0.75	0	0	0--0.78	0	0
DCDCPUB	PUB+DDR PHY	0.72	6.7	1.4	0.742	52	11.3
VDDQ	LPDDR4X MEM IO	0.62	2	0.4	0.64	25	4.7
VDDMEM	LPDDR4X MEM, WCN Digital in	1.112	6	2	1.134	52	17.3
...	...	...	...	...	...	...	...



# 功耗收益1/3

硬件功耗分解后，各测试方案及功耗收益如下表。

说明	加粗的优化项为当前可行方案	合入优化项置绿	
	历史测试方案，未使用的置灰		
优化分类	优化措施	720p预览收益/未使用方案原因	720p录像收益
Camera Sensor	驱动帧率从30fps调整为25fps	12mA，同步对比机配置，测试功耗前已合入，不计入总优化	-
音视频编码	720p, h264编码, 比特率2M; 录音采样率8k、比特率16k	同步对比机配置	-
	1080p, h264编码, 比特率4M; 录音采样率8k、比特率16k	同步对比机配置	-
	2160p, h264编码, 比特率8M; 录音采样率8k、比特率16k	同步对比机配置	-
Camera LWP	HAL层LWP架构功能简化: 手势识别、人脸检测、美颜、EIS、AI场景、场景识别	16mA	15mA
Camera硬件模块	3DNR_HW OFF	10mA，三方相机未启用此功能	-
	DCAM/CPP DVFS DPU降频到307.2M的基础上，VDD_MM降压到0.65v	7mA	7mA
	关闭video虚化调用NPU	8mA，三方相机未启用NPU	-
Camera TUNING	LSC 由隔3帧调整为隔5帧	无明显收益	-
	awb隔帧	3mA，有效果影响，评估不使用	-
	NR参数OFF	3mA，有效果影响，评估不使用	-

# 功耗收益2/3

续表。

优化分类		优化措施	720p预览收益	720p录像收益
Camera Power 策略	POWERHIT	三方camera应用打开camera时，CPU走cam_lowpower策略	88mA	117mA
		在cam_lowpower策略基础上，抑制CPU大小核提频积极性，小核-50，大核-60（后续客户按需修改提频积极性的幅度）	5mA	11mA
		抑制CPU大小核提频积极性小核-50，大核-60基础上，CPU小核限最高频768MHz+拔核(cpu4~cpu7)	9mA	6mA
		DDR固定1866	功耗高，评估不使用	-
		DDR固定768M	影响性能，评估不使用	-
	DFS	DDR 由camveryhigh freq 1866改为不设置DFS方案（720p预览场景主要跑在768m）	10mA	15mA
		DDR 由camveryhigh freq 1866 改为 camlow freq 1536	4mA，相比不设置DFS方案优化小，未采用	5mA
其他省电方案	其他电源	关闭不需要的电源域	13mA	15mA
Camera功耗可合入总优化			148mA	186mA

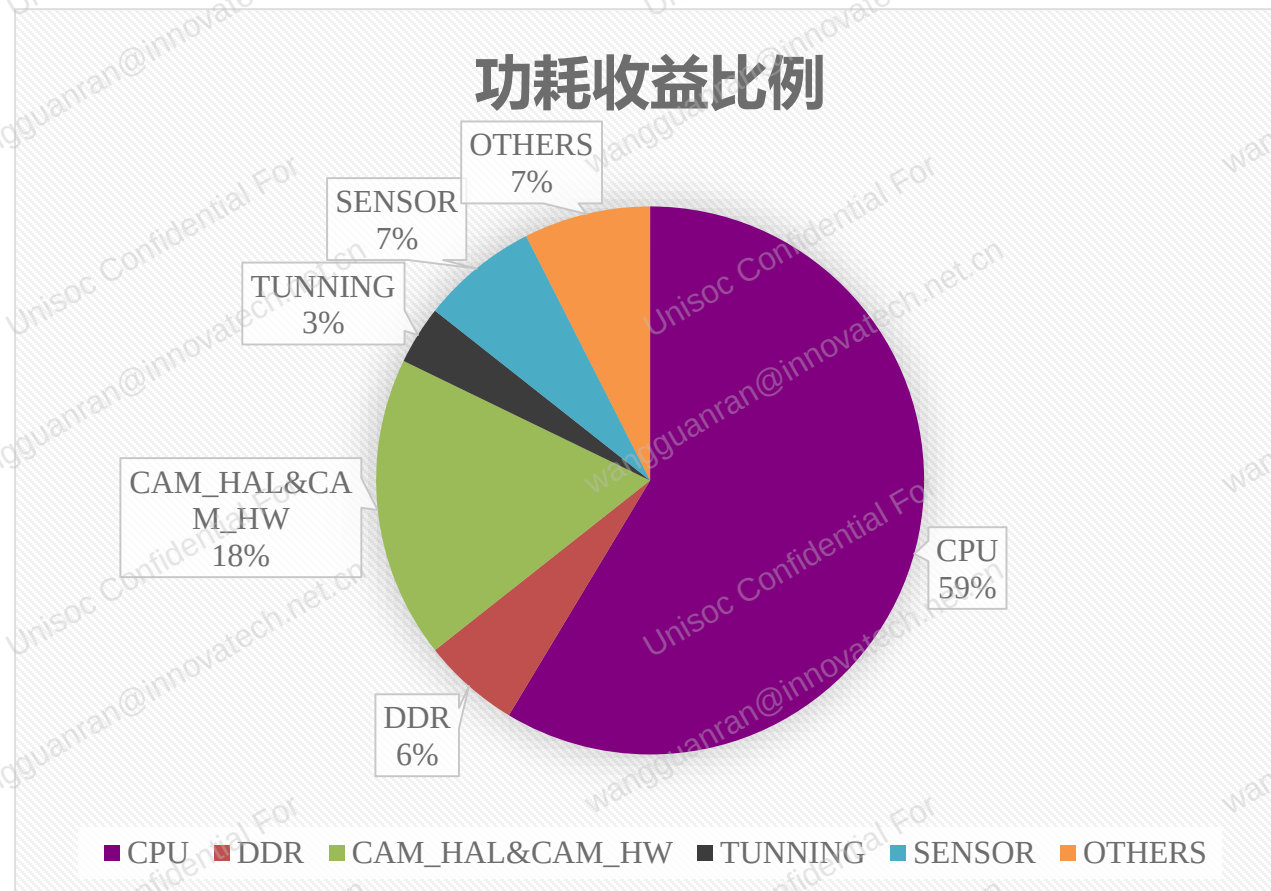
## 功耗收益3/3

从上述表格可汇总出如右功耗收益比例（右图仅供参考，不同平台、不同硬件环境、不同功能需求会导致差异）。首先要看CPU的工作情况，然后看HAL打开了哪些算法、调用了哪些硬件，再依序排查其他因素。

在经过上述优化后：

- 去除桌面电流的720P预览功耗，测试机功耗（142mA）已优于对比机（170mA）。
- 去除桌面电流的720P录像功耗，测试机功耗（163mA）已优于对比机（201mA）。

去除桌面电流	720P预览 (mA)	720P录像 (mA)
测试机-原始状态	290	349
测试机-优化后	142	163
对比机	170	201



05

# 参考文档

## 参考文档

1. 《功耗分析工具介绍》
2. 《Camera ADB命令使用指导手册》



06

附录

# Android 13设置功耗等级1/2

Android 13集中设置功耗等级位置: /vendor/sprd/modules/libcamera/hal3\_2v6/SprdCamera3OEMIf.cpp

```
int SprdCamera3OEMIf::getScenePerfLevel() {
 int level = 0;
 SPRD_DEF_Tag *sprddefInfo = mSetting->getSPRDDEFTagPTR();
 CONTROL_Tag controlInfo;
 mSetting->getCONTROLTag(&controlInfo);
 //for special 3rd app
 if (sprddefInfo->top_app_id == TOP_APP_WECHAT) {
 level = CAM_PERFORMANCE_LEVEL_7;
 } else if (isPerfSceneLevel()) {
 level = CAM_PERFORMANCE_LEVEL_6;
 } else if (isNormSceneLevel()) {
 level = CAM_PERFORMANCE_LEVEL_4;
 } else if (mSprdAppmodeId < 0) { //3rd
 level = CAM_PERFORMANCE_LEVEL_5;
 } else if (mRecordingMode == true) {
 #ifdef CONFIG_CAMERA_VIDEO_GRADING_POWERHINT_OPTIMIZE
 if (IS_1080P_SIZE(mVideoWidth, mVideoHeight) && mMultiCameraMode == MODE_SINGLE_CAMERA) {
 level = CAM_PERFORMANCE_LEVEL_11;
 } else if (IS_720P_SIZE(mVideoWidth, mVideoHeight) && mMultiCameraMode == MODE_SINGLE_CAMERA) {
 level = CAM_PERFORMANCE_LEVEL_10;
 } else
 level = CAM_PERFORMANCE_LEVEL_3;
 #elif defined(CONFIG_CAMERA_VIDEO_POWERHINT_OPTIMIZE)
 level = CAM_PERFORMANCE_LEVEL_9;
 #else
 level = CAM_PERFORMANCE_LEVEL_3;
 #endif
 } else if (getMultiCameraMode() != MODE_SINGLE_FACEID_UNLOCK) {
 level = CAM_PERFORMANCE_LEVEL_1;
 }
 return level;
}
```

# Android 13设置功耗等级2/2

不同分类包含的具体场景示例如下：

```
bool SprdCamera30EMIf::isPerfSceneLevel() {
 SPRD_DEF_Tag *sprddefInfo = mSetting->getSPRDDEFTagPTR();
 CONTROL_Tag controlInfo;
 mSetting->getCONTROLTag(&controlInfo);
 if (sprddefInfo->slowmotion > 1 || mHighFpsMode == true ||
 controlInfo.ae_target_fps_range[1] >= 90 ||
 sprddefInfo->top_app_id != 0 ||
 sprddefInfo->capture_mode > 1 ||
 getMultiCameraMode() == MODE_FOV_FUSION ||
 mDualVideoMode) {
 return true;
 }
 return false;
}

bool SprdCamera30EMIf::isNormSceneLevel() {
 SPRD_DEF_Tag *sprddefInfo = mSetting->getSPRDDEFTagPTR();
 if (getMultiCameraMode() == MODE_BLUR ||
 getMultiCameraMode() == MODE_BOKEH ||
 getMultiCameraMode() == MODE_PORTRAIT ||
 getMultiCameraMode() == MODE_PORTRAIT_SINGLE ||
 getMultiCameraMode() == MODE_PORTRAIT_SCENE ||
 mSprdAppmodeId == CAM_SCENE_MODE_PANORAMA ||
 mSprdAppmodeId == CAM_SCENE_MODE_NIGHT ||
 (sprddefInfo->sprd_filter_type > 0) ||
 (mRecordingMode && !mVideoWidth && !mVideoHeight)) {
 return true;
 }
 return false;
}
```

# Android 14设置功耗等级1/2

Android 14集中设置功耗等级位置：/vendor/sprd/modules/libcamera/hal3\_2v7/SprdCamera3Setting.cpp

```
int SprdCamera3Setting::getScenePerfLevel() {
 int level = 0;
 bool mRecordingMode = false;
 struct img_size streamSize;
 SPRD_DEF_Tag *sprddefInfo = getSPRDDEFtagPTR();
 CONTROL_Tag controlInfo;
 getCONTROLTag(&controlInfo);
 SessionParam sessionParam;
 memset(&sessionParam, 0, sizeof(SessionParam));
 memset(&streamSize, 0, sizeof(img_size));
 getPsParam(mCameraId, PREVIEW_TYPE, 0, SPRD_TAG_CONFIGSTREAM_INFO, &sessionParam);
 getPsParam(mCameraId, VIDEO_TYPE, 0, SPRD_TAG_RECORDING_MODE, &mRecordingMode);
 getPsParam(mCameraId, VIDEO_TYPE, 0, SPRD_TAG_STREAM_SIZE, &streamSize);
 //for special 3rd app
 if (sessionParam.mTopAppId != 0) {
 level = get3rdAppSceneLevel(sessionParam);
 } else if (isPerfSceneLevel(sprddefInfo, sessionParam, controlInfo)) {
 level = CAM_PERFORMANCE_LEVEL_6;
 } else if (isNormSceneLevel(sprddefInfo, mRecordingMode, streamSize, sessionParam)) {
 level = CAM_PERFORMANCE_LEVEL_4;
 } else if (IS_4K_SIZE(streamSize.width) && controlInfo.ae_target_fps_range[1] <= 30) {
 level = CAM_PERFORMANCE_LEVEL_5;
 } else if (mRecordingMode == true) {
 level = getVideoScenePerfLevel(streamSize);
 } else if (mMultiCameraMode != MODE_SINGLE_FACEID_UNLOCK) {
 level = CAM_PERFORMANCE_LEVEL_1;
 }
 HAL_LOGV("Powerhal:getScenePerfLevel:%d", level);
 return level;
}
```

Android 13上将原有的8个CAM\_PERFORMANCE\_LEVEL增加至11个，Android 14进一步增至14个。增加的Level是在原有的基础上将部分场景拆分出来进行配置，更加细化功耗的配置。

# Android 14设置功耗等级2/2

不同分类包含的具体场景示例如下：

```
bool SprdCamera3Setting::isPerfSceneLevel(SPRD_DEF_Tag *sprddefInfo, SessionParam mSessionParam, CONTROL_Tag controlInfo) {
 if (controlInfo.ae_target_fps_range[1] >= 90 ||
 sprddefInfo->capture_mode > 1 ||
 mMultiCameraMode == MODE_FOV_FUSION ||
 mSessionParam.appMode == CAMERA_MODE_DUAL_VIEW_MODE ||
 sprddefInfo->sprd_eis_enabled) {
 return true;
 }
 return false;
}

bool SprdCamera3Setting::isNormSceneLevel(SPRD_DEF_Tag *sprddefInfo, bool mRecordingMode,
 struct img_size streamSize, SessionParam mSessionParam) {
 if (mMultiCameraMode == MODE_BLUR ||
 mMultiCameraMode == MODE_BOKEH ||
 mMultiCameraMode == MODE_PORTRAIT ||
 mMultiCameraMode == MODE_PORTRAIT_SINGLE ||
 mMultiCameraMode == MODE_PORTRAIT_SCENE ||
 mSessionParam.sceneMode == CAM_SCENE_MODE_PANORAMA ||
 mSessionParam.sceneMode == CAM_SCENE_MODE_NIGHT ||
 (sprddefInfo->sprd_filter_type > 0) ||
 (mRecordingMode && !streamSize.width && !streamSize.height)) {
 return true;
 }
}
```



# 功耗等级对应的配置1/4

/vendor/sprd/modules/libcamera/ppm/camera\_power\_perf/SprdCameraPowerPerformance.cpp中，每个等级都会配置需要的CamPerformanceScene：

```
void SprdCameraSystemPerformance::setCamPerformanceScene(
 sys_performance_camera_scene camera_scene) {
 char ppm_debug[PROPERTY_VALUE_MAX] = {0};

 switch (camera_scene) {
 case CAM_PERFORMANCE_LEVEL_14:
 mapCamPerformanceScene(CAM_POWER_9863T_3RDAPP, mScene9863t3rdApp, CAM_VERYHIGH);
 break;
 case CAM_PERFORMANCE_LEVEL_13:
 if (!(strcmp("true", macro_ultrawidevideo)))
 mapCamPerformanceScene(CAM_POWER_ULTRAWIDE_VIDEO, mSceneUltrawideVideo, CAM_LOW);
 else
 mapCamPerformanceScene(CAM_POWER_LOWPPOWER_ON, mSceneLowPower, CAM_LOW);
 break;
```

## 功耗等级对应的配置2/4

/vendor/sprd/modules/libcamera/ppm/camera\_power\_perf/SprdCameraPowerPerformance.cpp中，每个等级都会配置需要的CamPerformanceScene：

```
void SprdCameraSystemPerformance::setCamPerformanceScene(
 sys_performance_camera_scene camera_scene) {
 char ppm_debug[PROPERTY_VALUE_MAX] = {0};

 case CAM_PERFORMANCE_LEVEL_6:
 if (!(strcmp("true", macro_lnk))) {
 mapCamPerformanceScene(CAM_POWER_HIGH_PERFORMANCE, mSceneHighPerformance, CAM_VERYHIGH);
 } else {
 mapCamPerformanceScene(CAM_POWER_PERFORMANCE_ON, mScenePerformance, CAM_VERYHIGH);
 }

 case CAM_PERFORMANCE_LEVEL_1:
 property_get("persist.vendor.cam.ppm.lowp", ppm_debug, "0");
 property_get("ro.debuggable", version, "1");
 if (!strcmp(ppm_debug, "1")) {
 mapCamPerformanceScene(CAM_POWER_LOWPPOWER_ON, mSceneLowPower, CAM_LOW);
 } else {
 if (!strcmp(version, "1")) {
 mapCamPerformanceScene(CAM_POWER_PERFORMANCE_ON, mScenePerformance, CAM_VERYHIGH);
 } else {
 mapCamPerformanceScene(CAM_POWER_LOWPPOWER_ON, mSceneLowPower, CAM_LOW);
 }
 }
 }
```

## 功耗等级对应的配置3/4

vendor/sprd/modules/libcamera/ppm/camera\_power\_perf/SprdCameraPowerPerformance.cpp中，每个CamPerformanceScene对应powerhintid关系如下：

```
void SprdCameraSystemPerformance::initPowerHint() {
 if (!mPowerManagerInit) {
 mPowerManager = new android::PowerHALManager();
 if (mPowerManager) {
 mPowerManager->init();
 mSceneLowPower = mPowerManager->createPowerHintScene(
 "camera", 0, "camera_lowpower");
 mScenePerformance =
 mPowerManager->createPowerHintScene("camera", 0, "camera_perf");
 mSceneSlightPerformance =
 mPowerManager->createPowerHintScene("camera", 0, "camera_lowpower_1");
 mSceneHighPerformance =
 mPowerManager->createPowerHintScene("camera", 0, "camera_high_perf");
 mSceneSpecialApp =
 mPowerManager->createPowerHintScene("camera", 0, "camera_SpecialApp");
 mSceneVideo720P =
 mPowerManager->createPowerHintScene("camera", 0, "camera_video_720p");
 mSceneVideo1080P =
 mPowerManager->createPowerHintScene("camera", 0, "camera_video_1080p");
 mSceneCtsPerf =
 mPowerManager->createPowerHintScene("camera", 0, "camera_cts_perf");
 mSceneLowPowerVideo =
 mPowerManager->createPowerHintScene("camera", 0, "camera_lowpower_video");
 }
 }
}
```

例如，mScenePerformance对应实际的功耗场景为“camera\_perf”。

## 功耗等级对应的配置4/4

每个powerhintid实际配置参数的位置:

/vendor/sprd/modules/power/vendor-power/config\_files/qogirl6/mfeature/kernel/kernel5.15/power\_scene\_ocnfig.xml

注意: 根据实际的项目确认对应的文件。

```
<scene name="camera_high_perf" >
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_min_freq" value="1820000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_max_freq" value="1820000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy6" file="scaling_min_freq" value="2028000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy6" file="scaling_max_freq" value="2028000" />
</scene>
<scene name="camera_perf" >
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_min_freq" value="1820000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy0" file="scaling_max_freq" value="1820000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy6" file="scaling_min_freq" value="2028000" />
 <set path="/sys/devices/system/cpu/cpufreq/policy6" file="scaling_max_freq" value="2028000" />
</scene>
<scene name="camera_lowpower" >
 <set path="/sys/devices/system/cpu/cpufreq/policy0/uscfreq" file="freq_margin" value="80" />
 <set path="/sys/devices/system/cpu/cpufreq/policy6/uscfreq" file="freq_margin" value="0" />
</scene>
```

主要配置CPU的参数, 如频点、升频积极性等。



# Userdebug版本测试注意事项

如需使用Userdebug版本测试方案优化时，需注意：

- **关闭ylog、设置log等级。**

- 关闭ylog：拨号盘输入”\*##83781#\*#\*”（如无拨号盘，请参考[其他TIPS](#)使用adb命令进入），最上方一行选择**DEBUG&LOG > YLOG**，点击屏幕下方的**STOP**按钮（如按钮显示**START**则不用管）
- 设置log等级：adb shell setprop persist.vendor.cam.mlog.loglevel 333333

- **配置Camera功耗测试场景。**

拨号盘输入”\*##83781#\*#\*”（如无拨号盘，请参考[其他TIPS](#)使用adb命令进入），最上方一行选择**HARDWARE**，关闭Camera Idle Sleep，开启Camera Power Test Mode。

- 如果只是测一颗摄像头的功耗，请开启Mutl Camera Bypass Mode。
- 如果要测双摄模式，请关闭Mutl Camera Bypass Mode。

- **关闭蓝牙、Wi-Fi、NFC、屏幕亮度自动调节、位置信息，开启飞行模式。**

- **屏幕亮度调到最低，屏幕超时改为常亮/最长。**





# 添加TOPAPP

## Android 13

```
①vendor/sprd/modules/libcamera/hal3_2v6/SprdCamera3Setting.cpp
162 static TopAppS topAppList[] = {
163 { TOP_APP_NONE, "com.android.camera2" },
164 { TOP_APP_WECHAT, "com.tencent.mm" },
165 // { TOP_APP_WTFACTORY, "" },
166 { TOP_APP_QQ, "com.tencent.mobileqq" },
167 { TOP_APP_FACEBOOK, "com.facebook.katana" },
168 { TOP_APP_INSTAGRAM, "com.instagram.android" },
169 { TOP_APP_MESSENGER, "com.facebook.orca" },
170 { TOP_APP_SNAPCHAT, "com.snapchat.android" },
171 { TOP_APP_WHATSAPP, "com.whatsapp" },
172 { TOP_APP_QQINT, "com.tencent.mobileqqi" },
173 { TOP_APP_READYAPP, "com.google.android.apps.tachyon.readyapp" },
174 { TOP_APP_CAMERAGO, "com.google.android.apps.cameralite" },
>+++ { TOP_APP_TEST3RD, "xxxxxxx" }, // 这里的xxxxxxx代指app的包名
175 };
```

```
②vendor/sprd/modules/libcamera/common/inc/cmr_common.h
1833 enum top_app_id {
1834 //set other thirdparty apps besides the list to -1
1835 TOP_APP_NONE = 0,
1836 TOP_APP_WECHAT = (1 << 0),
1837 TOP_APP_WTFACTORY = (1 << 1),
1838 TOP_APP_QQ = (1 << 2),
1839 TOP_APP_FACEBOOK = (1 << 3),
1840 TOP_APP_INSTAGRAM = (1 << 4),
1841 TOP_APP_MESSENGER = (1 << 5),
1842 TOP_APP_SNAPCHAT = (1 << 6),
1843 TOP_APP_WHATSAPP = (1 << 7),
1844 TOP_APP_QQINT = (1 << 8),
1845 TOP_APP_READYAPP = (1 << 9),
1846 TOP_APP_CAMERAGO = (1 << 10),
>+++ TOP_APP_TEST3RD = (1 << 11),
```

## Android 14

```
①vendor/sprd/modules/libcamera/idispatch/android/Camera3Adapter.cpp
48 static TopAppS topAppList[] = {
49 { TOP_APP_NONE, "com.android.camera2" },
50 { TOP_APP_WECHAT, "com.tencent.mm" },
51 // { TOP_APP_WTFACTORY, "" },
52 { TOP_APP_QQ, "com.tencent.mobileqq" },
53 { TOP_APP_FACEBOOK, "com.facebook.katana" },
54 { TOP_APP_INSTAGRAM, "com.instagram.android" },
55 { TOP_APP_MESSENGER, "com.facebook.orca" },
56 { TOP_APP_SNAPCHAT, "com.snapchat.android" },
57 { TOP_APP_WHATSAPP, "com.whatsapp" },
>+++ { TOP_APP_TEST3RD, "xxxxxxx" }, // 这里的xxxxxxx代指app的包名
```

```
②vendor/sprd/modules/libcamera/common/inc/cmr_common.h
2233 //for some thirdparty camera apps
2234 TOP_APP_MTX = 30,
2235 TOP_APP_MEIYAN,
2236 TOP_APP_FACEU,
2237 TOP_APP_LITE,
2238 TOP_APP_WUTA,
2239 TOP_APP_DOUBTNUT,
2240 TOP_APP_CALI,
2241 TOP_APP_TWITTER,
>+++ TOP_APP_TEST3RD
```



# LSC隔帧-IspTool

ALSC

Alsc\_Calibration Basic\_Alsc Post\_Gain Flash Adaptive\_Alsc

Control

Alsc\_enable: On Bypass\_otp: Off use\_common\_param: On

Calc\_freq: 3 iir\_weight: 10 Load\_level: normal

init\_lsc\_index: 3 fix\_table\_index: 9

slsc

Alg\_mode: 5 Show Smart LSC

Color.Compensation

Comp.Range

Level R: 10

Level B: 10

Comp.Strength

Weight R: 40

Weight B: 40

Save[S]

Scenario id Scenario name Image size

0x0000000000000000	com	2688x1512
0x0000010000000000	prv0	2688x1512
0x0000050000000000	cap0	2688x1512
0x0000090000000000	video0	2688x1512

BLOCK ISP

RGB GAMMA

RGBLTM

Y GAMMA

ATM4GAMMA

Color

CCM

HSV

ALSC

AWB

3D\_LUT

Focus

RGB\_AFM

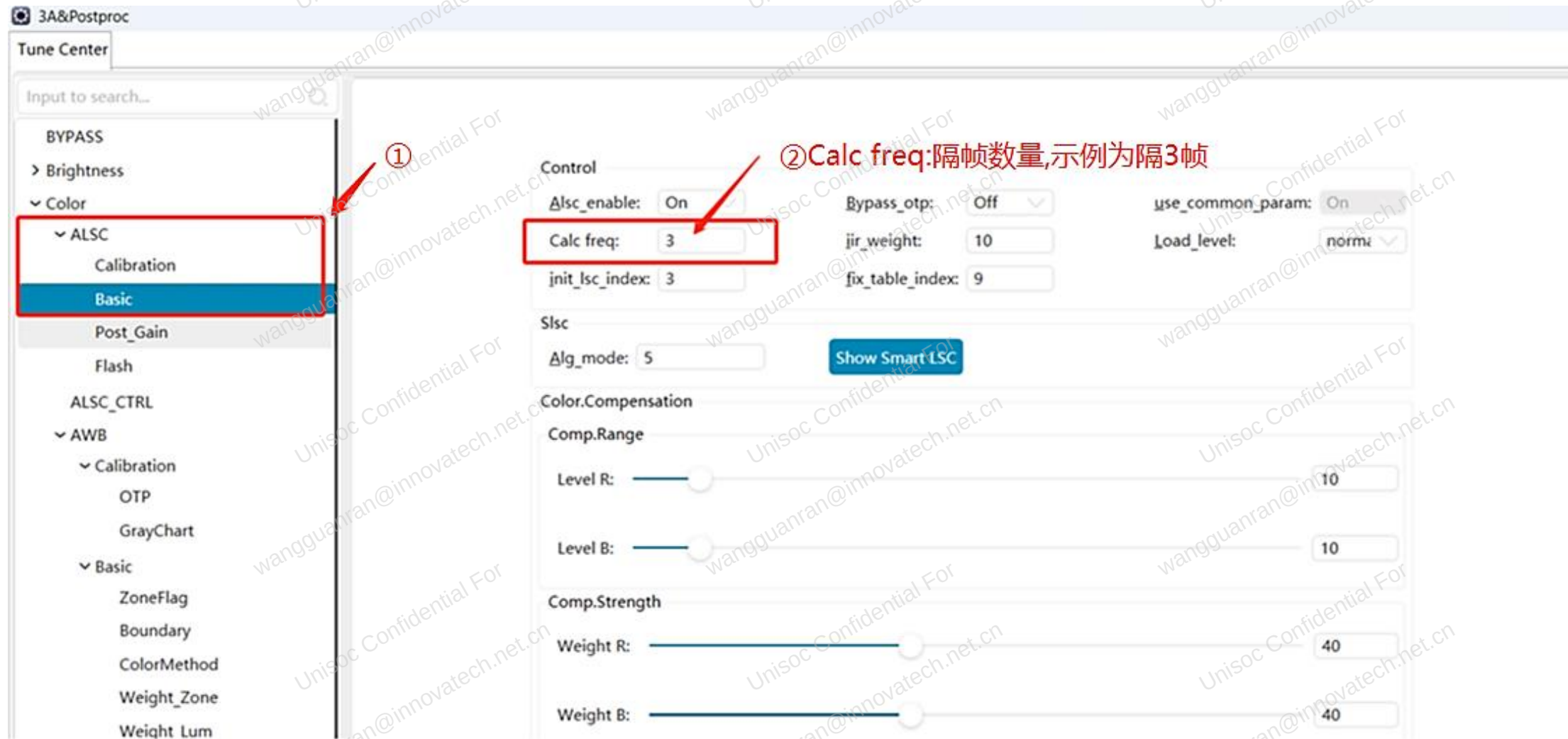
AF

Nr

② Calc\_freq: 隔帧数量, 示例为隔3帧

①

# LSC隔帧-ViviMagic



# AWB隔帧-IspTool

AWB

Parameter\_List

- Calibration\_AWB
- Basic\_AWB
- Advance\_AWB
- Setting\_AWB
  - MWB
  - Others

Calculation Interval:

stat memory size:

Init Gain

x:  y:

Stat\_filter

mode:

Awb Smooth

Flag:

Dynamic\_smooth

BV:

Delete

Ins

BV1	BV2	BV3	BV4	BV5	BV6
-200	0	200	1000		

Smooth\_Frame\_num:

②隔帧数量, 0代表不隔帧, 1代表隔1帧

100%

Scenario id	Scenario name	Image siz
0x0000000000000000	com	2688x151
0x0000010000000000	prv0	2688x151
0x0000050000000000	cap0	2688x151
0x0000090000000000	video0	2688x151

BLOCK | ISP

Y GAMMA

ATM4GAMMA

Color

CCM

HSV

ALSC

AWB

3D\_LUT

Focus

RGB\_AFM

AF

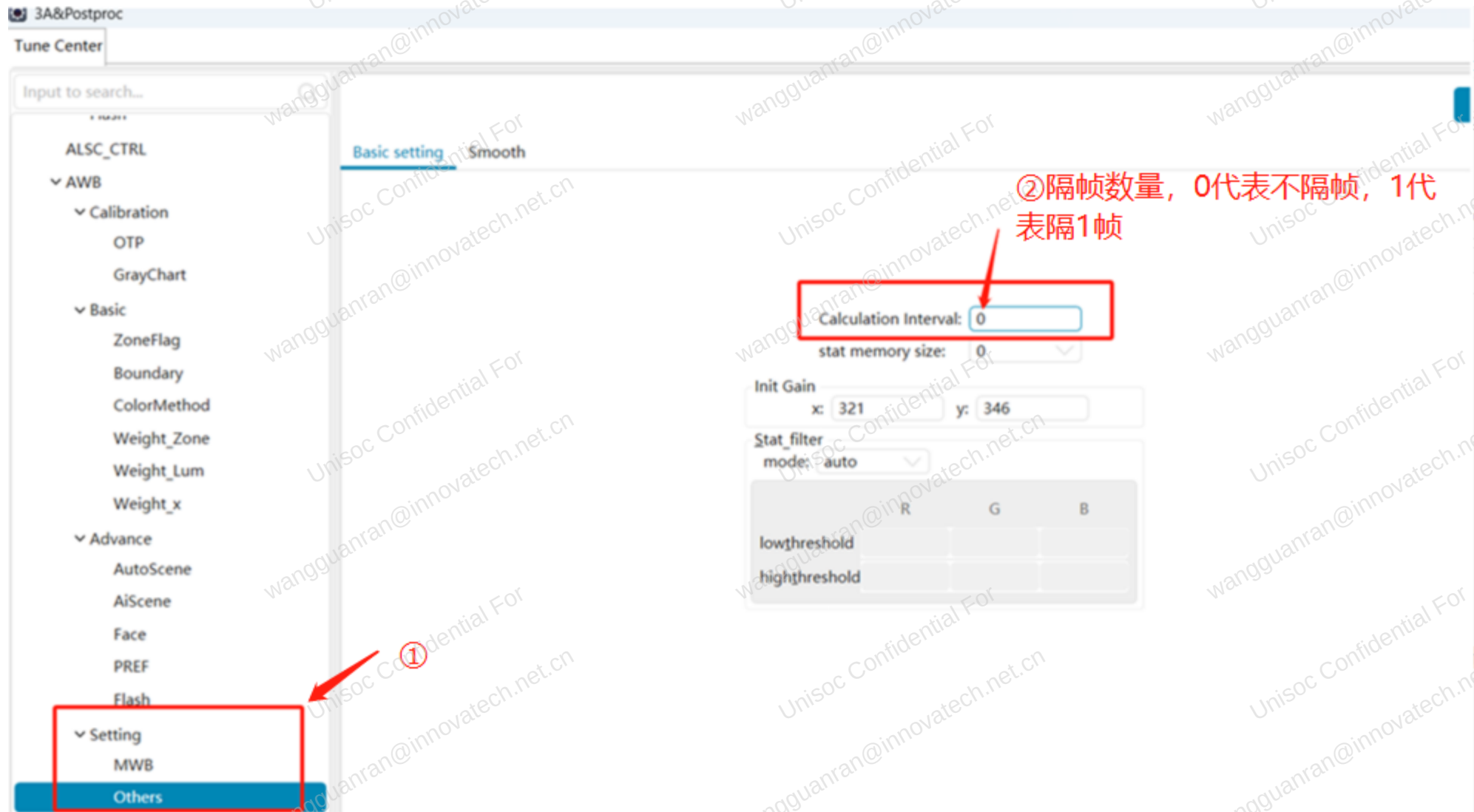
Nr

NR

Postproc And Other

①

# AWB隔帧-ViviMagic





## 其他TIPS

- 无拨号键时可通过以下命令进入工程模式（拨号键\*##83781#\*##\*）：

adb shell am start -n com.sprd.engineermode/com.sprd.engineermode.EngineerModeActivity 3

- User版本上快速打开debug log打印方法

进入工程模式，在最上方一行选择**DEBUG&LOG > Dynamic Log > Camera Log**，就会有常规camera模块的log开关，包括app、hal、性能功耗测试、3A。Camera HAL可以选择 hal、oem、IspMw、Sensor的log等级，建议选level 4或5，重启手机生效。



# 谢谢



本文件所含数据和信息都属于紫光展锐（上海）科技股份有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

紫光展锐 UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA及其他紫光展锐的商标均为紫光展锐（上海）科技股份有限公司及/或其子公司、关联公司所有。本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。